

Modelos de Difusão

Experimento- U-net

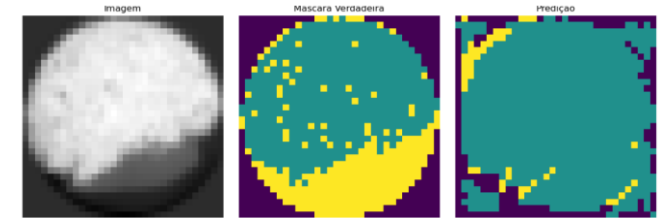
CPE883 - Tópicos Especiais em Machine Learning
Vivian Rodrigues (DRE:125228569)

Avanços no Experimento 3

Segmentação com Convoluções Reais KAN

- Primeiro modelo com convoluções reais (CKANConv2D)
- Uso de **KAN_Convolutional_Layer** com splines aprendíveis 2D
- Treinamento completo em múltiplas épocas com **visualização das predições**

- Redução da resolução para **128×128 → 32×32** (limites de GPU)



- Saída multiclasse com logits para **3 classes** (rocha, poro, fundo)
- Modelo inicial ainda simples, mas **já segmenta estruturas geológicas**
- Base para evolução futura para **U-Net com CKAN convolucional**

Proximas etapas:

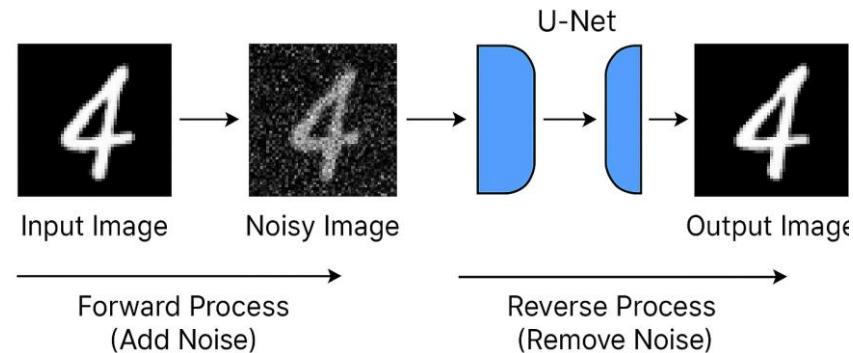
Aumentar a resolução das imagens de entrada (de 32×32 para 128×128).

Adicionar métricas quantitativas (IoU, Dice Score).

Avaliar generalização com separação treino/validação/teste.

Resumo do Projeto: Diffusion com MNIST

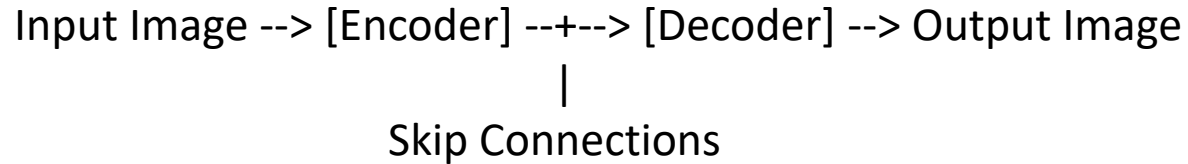
- Objetivo: Treinar um modelo de Diffusion para gerar dígitos manuscritos (MNIST).
- Pipeline simples usando modelo U-Net + Gaussian Diffusion.



- Experimento inicial com código pronto para treino.
- Código baseado na implementação: denoising-diffusion-pytorch (<https://github.com/lucidrains/denoising-diffusion-pytorch>).

Arquitetura U-Net Simplificada

- U-Net = Encoder + Decoder + Skip onnections.



- Encoder extrai características multi-escala da imagem.
- Decoder reconstrói a imagem a partir dessas características.
- Skip connections preservam detalhes espaciais importantes.

Pipeline do Experimento

- 1. Baixa MNIST e exporta imagens para ./mnist_images.
- 2. Inicializa U-Net + GaussianDiffusion.
- 3. Cria Trainer com imagens e configurações.
- 4. Salva amostras em ./results_mnist a cada 500 steps.

Código Principal - Treino e Geração

```
# Inicializa modelo e diffusion
model = Unet(dim=32, dim_mults=(1,2,4), channels=1)
diffusion = GaussianDiffusion(
    model, image_size=32, timesteps=1000,
    sampling_timesteps=100, objective='pred_v'
)

# Cria trainer
trainer = Trainer(
    diffusion, folder='./mnist_images',
    train_batch_size=64, train_num_steps=5000,
    save_and_sample_every=500,
    results_folder='./results_mnist',
    calculate_fid=False
)

trainer.train()
```

Próximos passos

- Concluir Experimento com MNIST
- Substituir MNIST por seu dataset de microtomografia
 - Adaptar o `export_mnist_to_folder` para salvar imagem / máscara.
 - Tentar manter as imagens redimensionadas (ex: 128x128).
 - Criar uma versão do `train.py` que use imagens reais + máscaras
- Usar Diffusion condicional com pares (imagem → máscara)
 - Opção 1: Diffusion supervisionado (com U-Net padrão)
 - Opção 2: Usar modelo já existente (ex: <https://arxiv.org/abs/2303.09487>)
- Experimento Mínimo: Imagem → Máscara com Diffusion

Referências

- Denoising Diffusion Probabilistic Models: <https://arxiv.org/abs/2006.11239>
- denoising-diffusion-pytorch - lucidrains: <https://github.com/lucidrains/denoising-diffusion-pytorch>
- PyTorch-FID: <https://github.com/mseitzer/pytorch-fid>